

```
In [5]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

# Load the dataset
file_path = "indian_food.csv"
df = pd.read_csv(file_path)

# Step 1: Replace '-1' in 'state' with the mode
mode_state = df[df['state'] != '-1']['state'].mode()[0]
df['state'] = df['state'].replace('-1', mode_state)

# Step 2: Fill corresponding region where 'state' was replaced
mode_region = df[df['state'] == mode_state]['region'].mode()[0]
df['region'] = df['region'].replace('-1', mode_region)

# Step 3: Replace '-1' in 'cook_time' with the mean
mean_cook_time = df[df['cook_time'] != -1]['cook_time'].astype(float).mean()
df['cook_time'] = df['cook_time'].replace(-1, mean_cook_time)

# 1. Pie Chart: Most preferred flavors
flavor_counts = df['flavor_profile'].value_counts()
plt.figure(figsize=(8, 6))
plt.pie(flavor_counts, labels=flavor_counts.index, autopct='%1.1f%%', colors=sns.color_palette("pastel"))
plt.title("Most Preferred Flavors by Customers")
plt.show()

# 2. Bar Chart: Most requested courses
course_counts = df['course'].value_counts()
plt.figure(figsize=(10, 6))
ax = sns.barplot(x=course_counts.index, y=course_counts.values, palette="viridis")

# Annotate values on top of bars
for i, v in enumerate(course_counts.values):
    ax.text(i, v + 1, str(v), ha='center', fontsize=12)

plt.title("Most Requested Courses by Customers", fontsize=14)
plt.xlabel("Course", fontsize=12)
```

```
plt.ylabel("Count", fontsize=12)
plt.xticks(rotation=45)
plt.show()

# 3. Grouping and plotting
veg_counts = df.groupby(['region', 'Diet']).size().unstack()

# Plot the stacked bar chart with custom colors
veg_counts.plot(kind='bar', stacked=True, figsize=(10, 6), color=['green', 'red'])

# Titles and labels
plt.title("Comparison of Vegetarian and Non-Vegetarian Orders by Region")
plt.xlabel("Region")
plt.ylabel("Count")
plt.legend(title="Diet", labels=["Vegetarian", "Non-Vegetarian"])
plt.show()

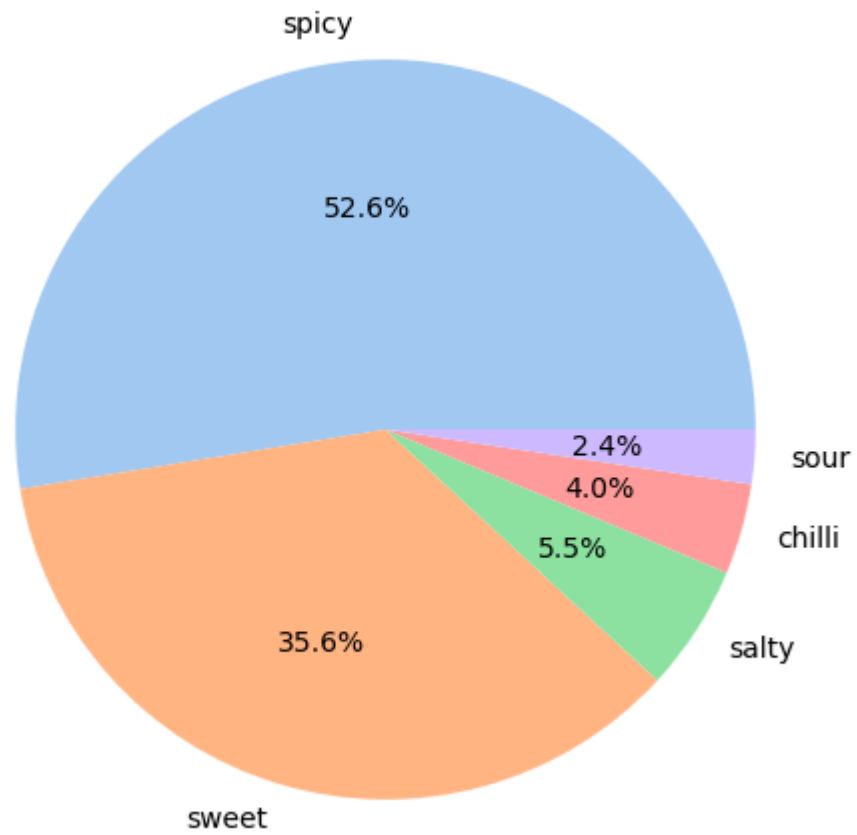
# 4. Box Plot: Cooking Time Distribution
plt.figure(figsize=(10, 6))
sns.boxplot(y=df['cook_time'], palette="coolwarm")
plt.title("Distribution of Cooking Time with Outliers")
plt.ylabel("Cooking Time (minutes)")
plt.show()

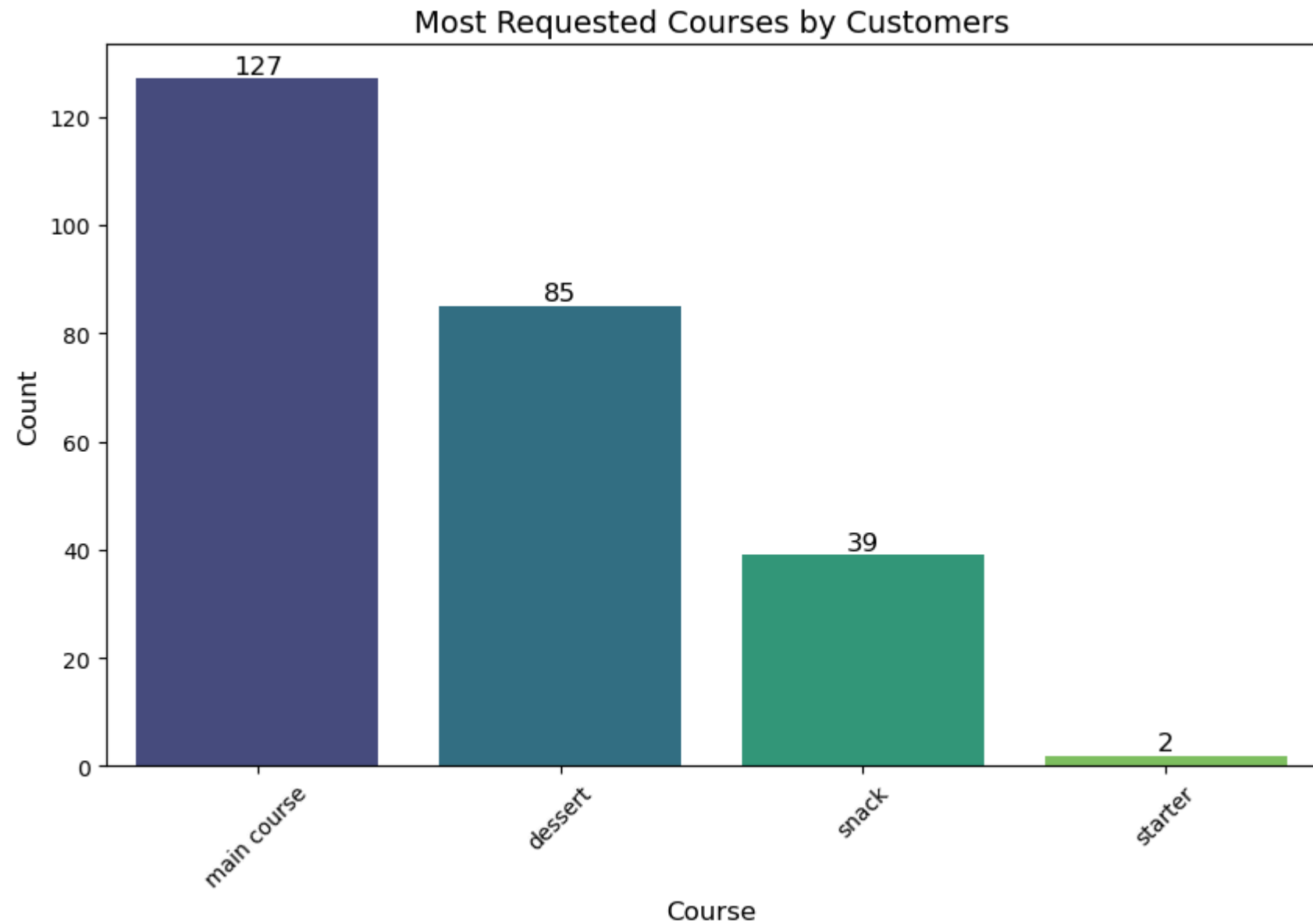
# Checking for outliers
Q1 = df['cook_time'].quantile(0.25)
Q3 = df['cook_time'].quantile(0.75)
IQR = Q3 - Q1

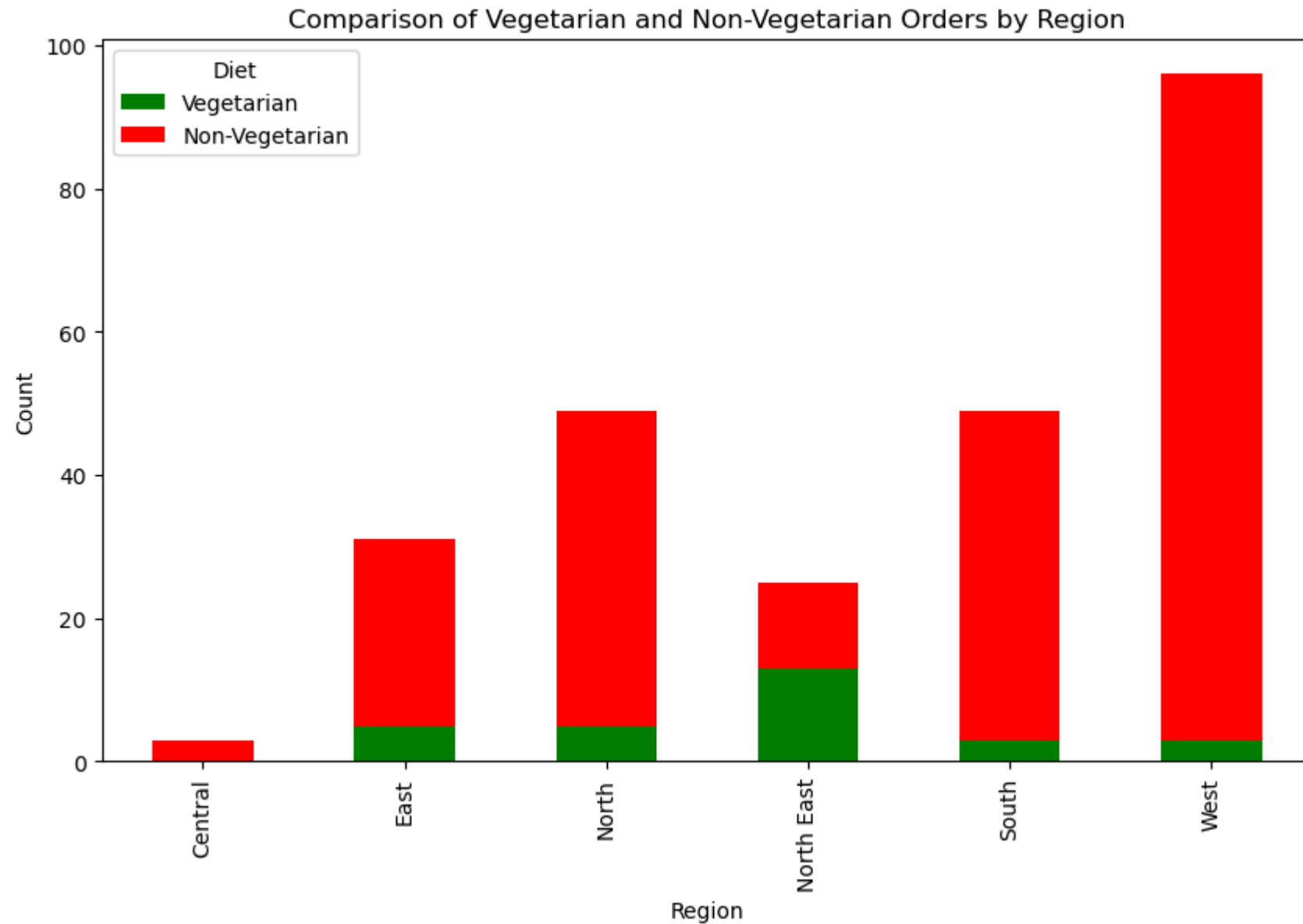
outliers = df[(df['cook_time'] < Q1 - 1.5 * IQR) | (df['cook_time'] > Q3 + 1.5 * IQR)]

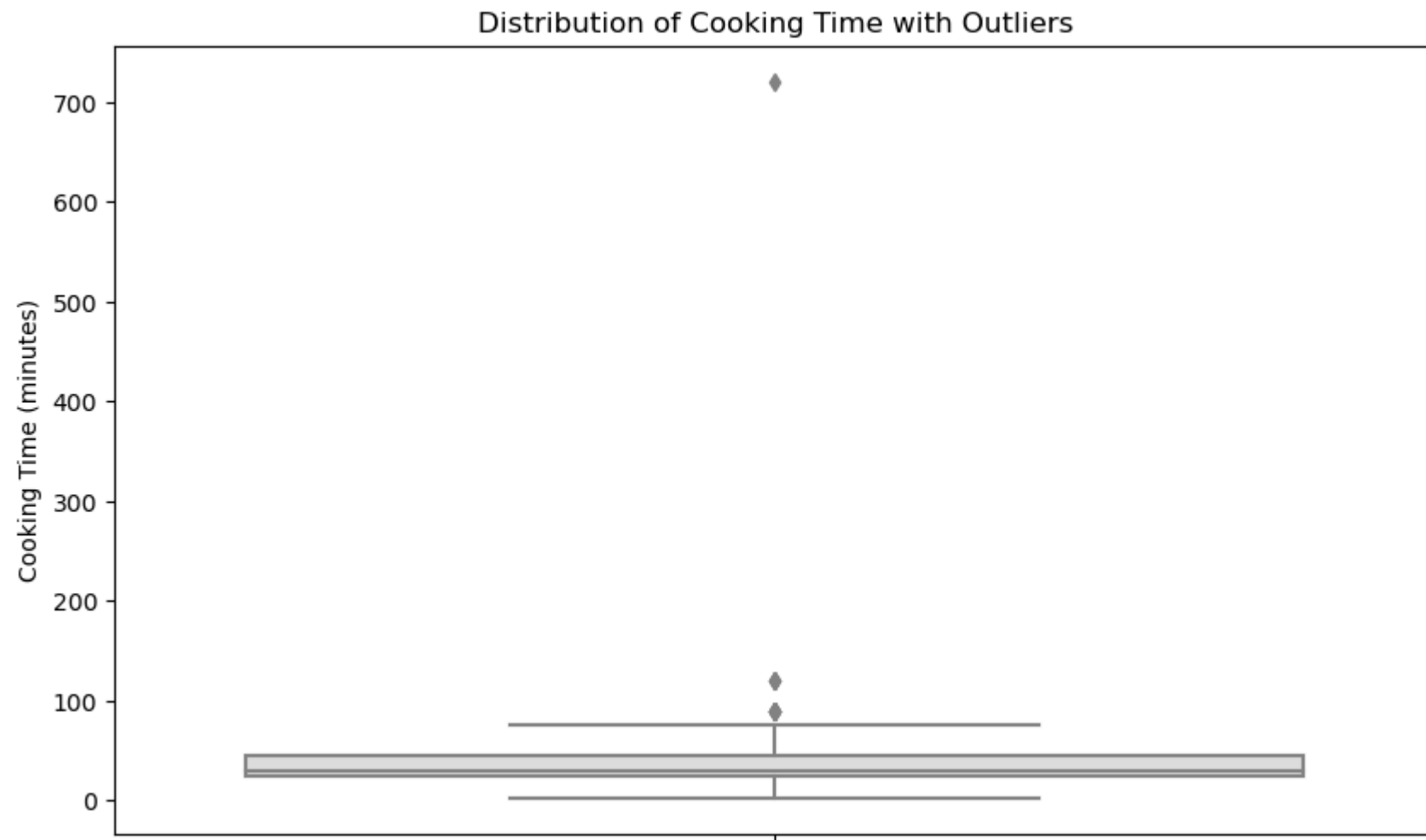
print(f'Number of outliers detected: {len(outliers)}')
```

Most Preferred Flavors by Customers









Number of outliers detected: 11

```
In [38]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
file_path = "Billionaires_Stats.csv"

df = pd.read_csv(file_path)
```

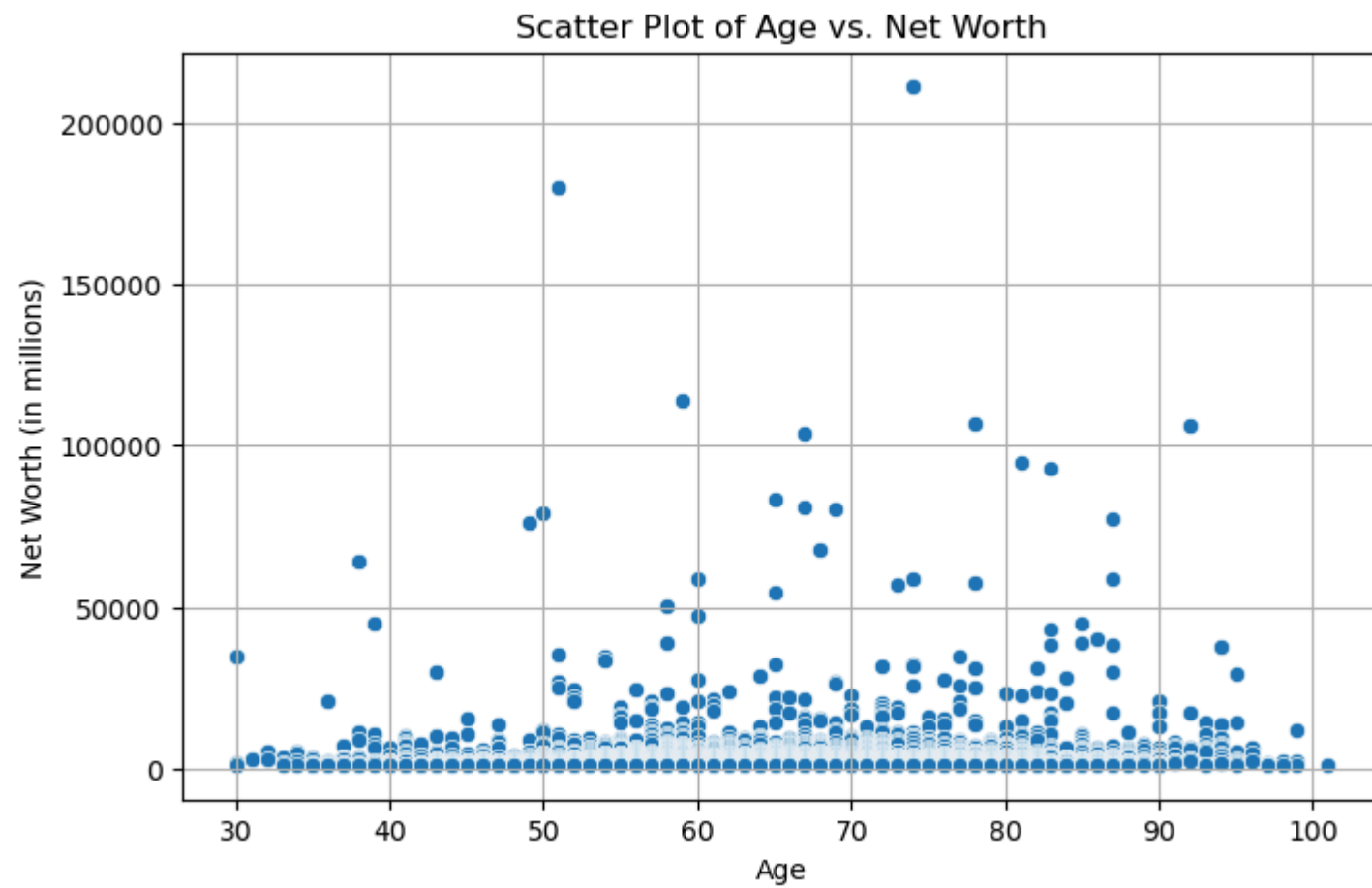
```
# Filter data for billionaires aged 30 or older
df_filtered = df[df["age"] >= 30]

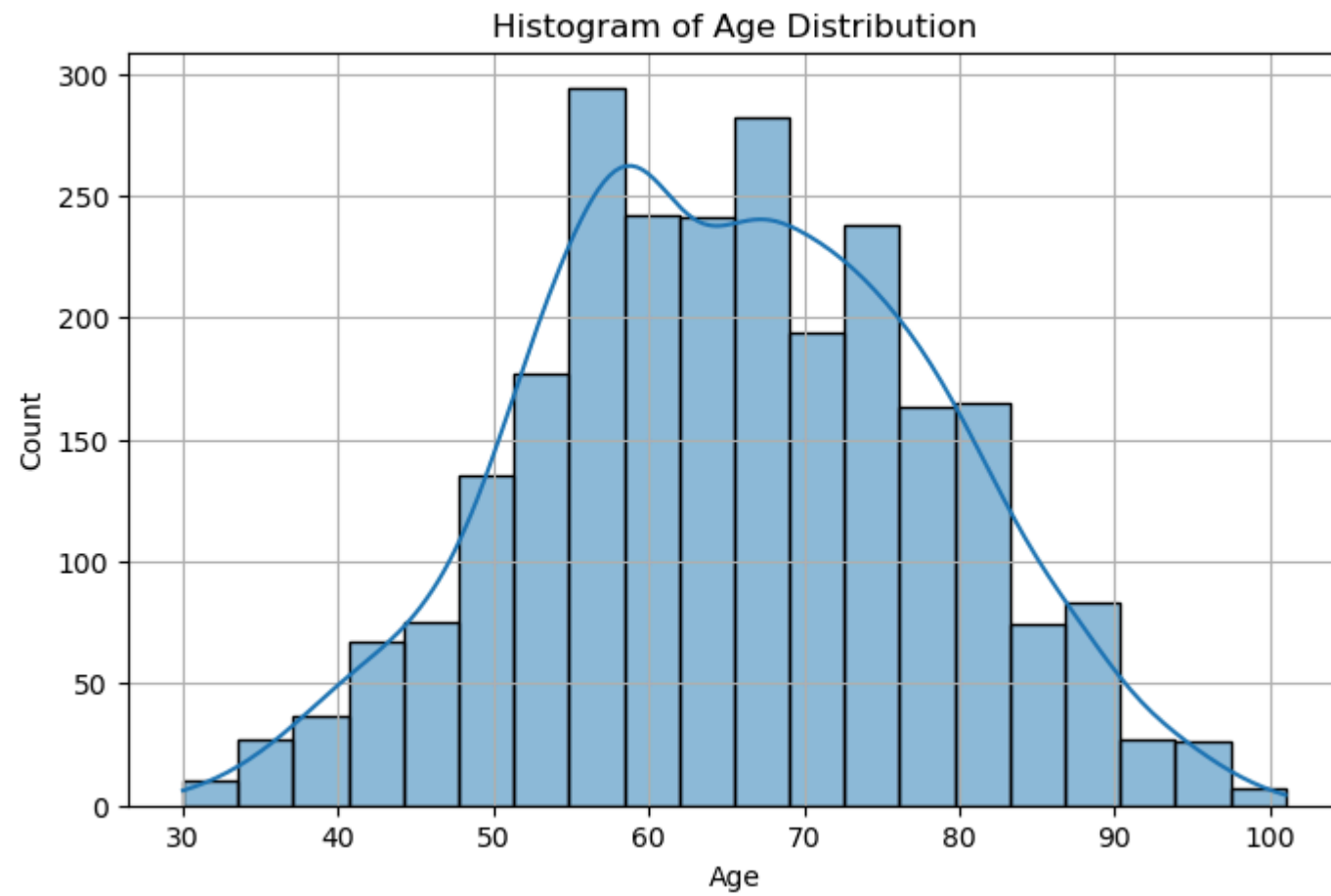
# Scatter plot: Age vs. Net Worth
plt.figure(figsize=(8, 5))
sns.scatterplot(x=df_filtered["age"], y=df_filtered["finalWorth"], edgecolor='w')
plt.xlabel("Age")
plt.ylabel("Net Worth (in millions)")
plt.title("Scatter Plot of Age vs. Net Worth")
plt.grid(True)
plt.savefig("scatter_plot.png")
plt.show()

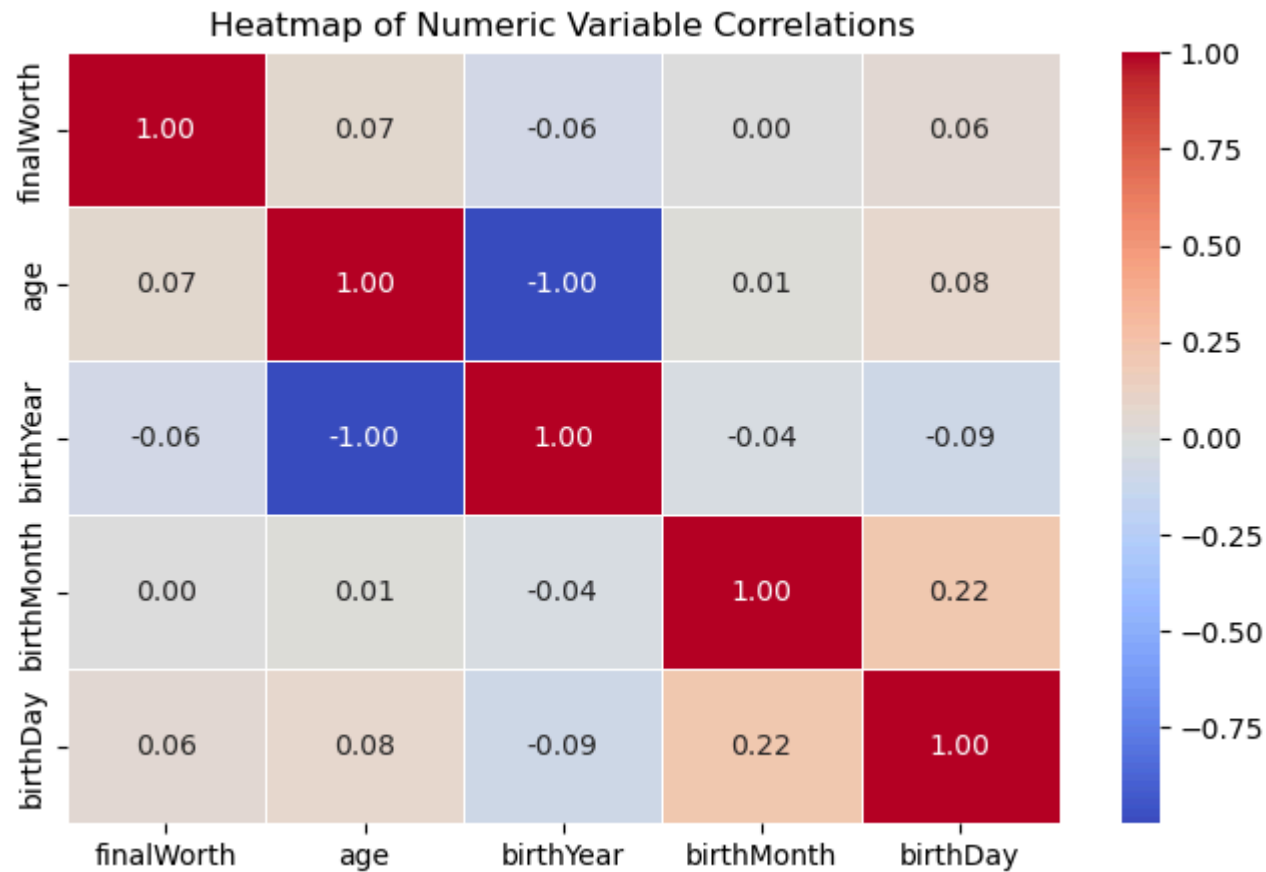
# Histogram for age distribution
plt.figure(figsize=(8, 5))
sns.histplot(df_filtered["age"], bins=20, kde=True)
plt.xlabel("Age")
plt.ylabel("Count")
plt.title("Histogram of Age Distribution")
plt.grid(True)
plt.savefig("age_histogram.png")
plt.show()

# Compute the correlation matrix
correlation_matrix = df_filtered.corr(numeric_only=True)

# Create a heatmap with correlation values
plt.figure(figsize=(8, 5))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Heatmap of Numeric Variable Correlations")
plt.show()
plt.savefig("heatmap.png")
print("Heatmap saved as 'heatmap.png' successfully!")
```







Heatmap saved as 'heatmap.png' successfully!

<Figure size 640x480 with 0 Axes>

```
In [32]: # 5 types of charts are:
#1.Line Chart (Matplotlib): Used to show trends over time (e.g., stock prices, temperature changes).
#2.Bar Chart (Seaborn): Used to compare categorical data (e.g., sales by product, revenue by region)
#3.Pie Chart (Plotly): Displays proportions of a whole (e.g., market share, budget distribution).
#4.Scatter Plot (Matplotlib): Shows relationships between two numerical variables (e.g., height vs. weight).
#5.Histogram (Seaborn): Displays the distribution of a dataset (e.g., test scores, age distribution).

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import matplotlib.cm as cm # Import the color map module

# Load the data
file_path = "Car.xlsx" # Ensure the correct file path
car_data = pd.read_excel(file_path)

# Step 1: Data Cleaning
# Drop rows with any missing values
car_data.dropna(inplace=True)

# Ensure correct data types
car_data['Year'] = car_data['Year'].astype(int)
car_data['Engine_Size'] = car_data['Engine_Size'].astype(float)
car_data['Mileage'] = car_data['Mileage'].astype(int)
car_data['Price'] = car_data['Price'].astype(float)

# Step 2: Histogram (Distribution of Prices)
plt.figure(figsize=(10, 6))
plt.hist(car_data['Price'], bins=20, color='skyblue', edgecolor='black')
plt.title('Histogram of Car Prices')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()

# Step 3: Bar Chart (Average Price by Brand)
avg_price_by_brand = car_data.groupby('Brand')['Price'].mean().sort_values(ascending=False)

# Normalize the prices for color mapping
norm = plt.Normalize(avg_price_by_brand.min(), avg_price_by_brand.max())

# Choose a color map
cmap = cm.get_cmap('viridis')

# Map the normalized prices to colors
colors = cmap(norm(avg_price_by_brand.values))

# Create the bar chart (Average Price by Brand)
plt.figure(figsize=(12, 8))
avg_price_by_brand.plot(kind='bar', color=colors)
plt.title('Average Price by Brand')
plt.xlabel('Brand')
```

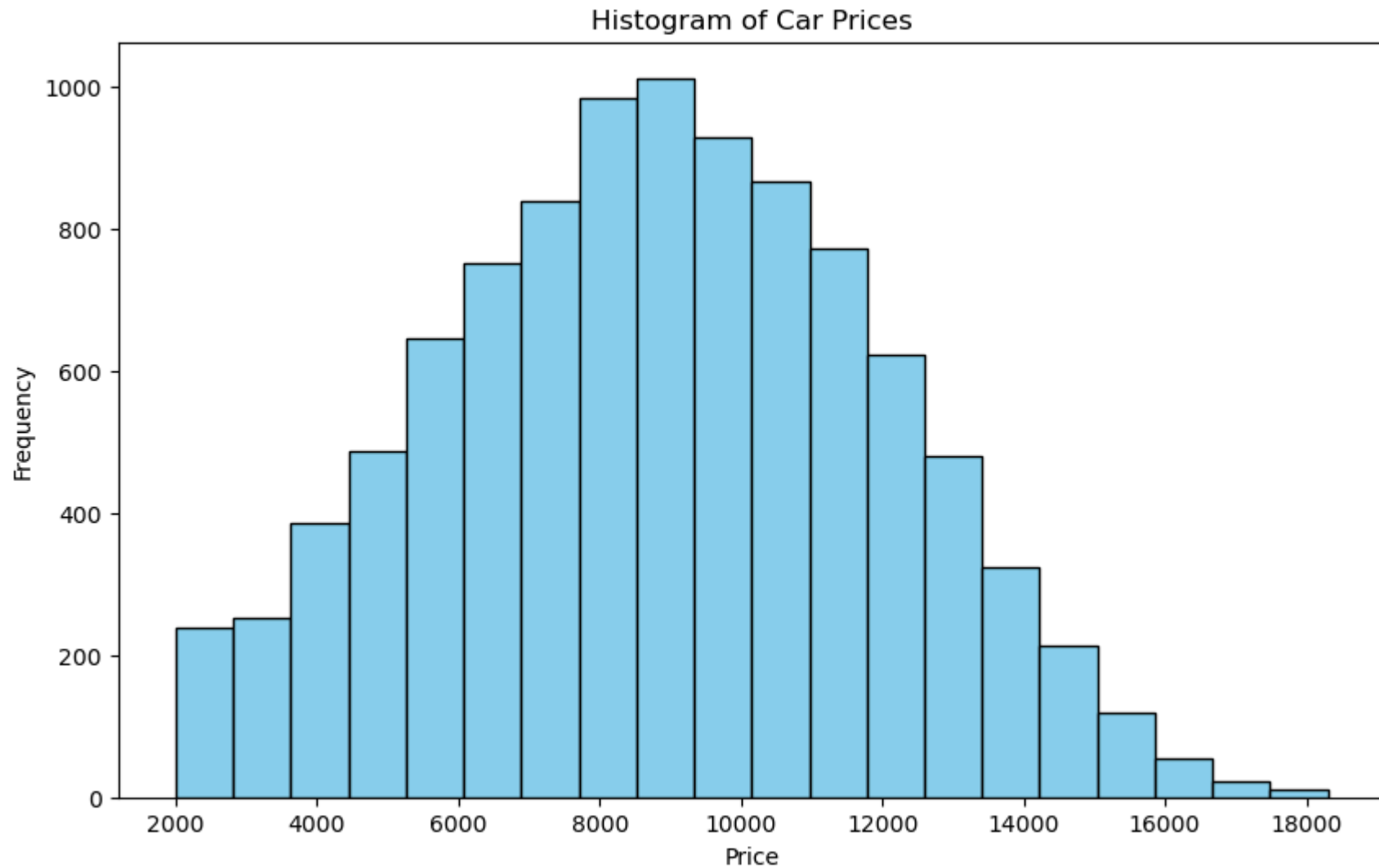
```
plt.ylabel('Average Price')
plt.xticks(rotation=45)
plt.show()

# Step 4: Scatter Plot (Price vs Mileage)
plt.figure(figsize=(10, 6))
sns.scatterplot(data=car_data, x='Mileage', y='Price', hue='Fuel_Type')
plt.title('Scatter Plot of Price vs Mileage')
plt.xlabel('Mileage')
plt.ylabel('Price')
plt.legend(title='Fuel Type')
plt.show()

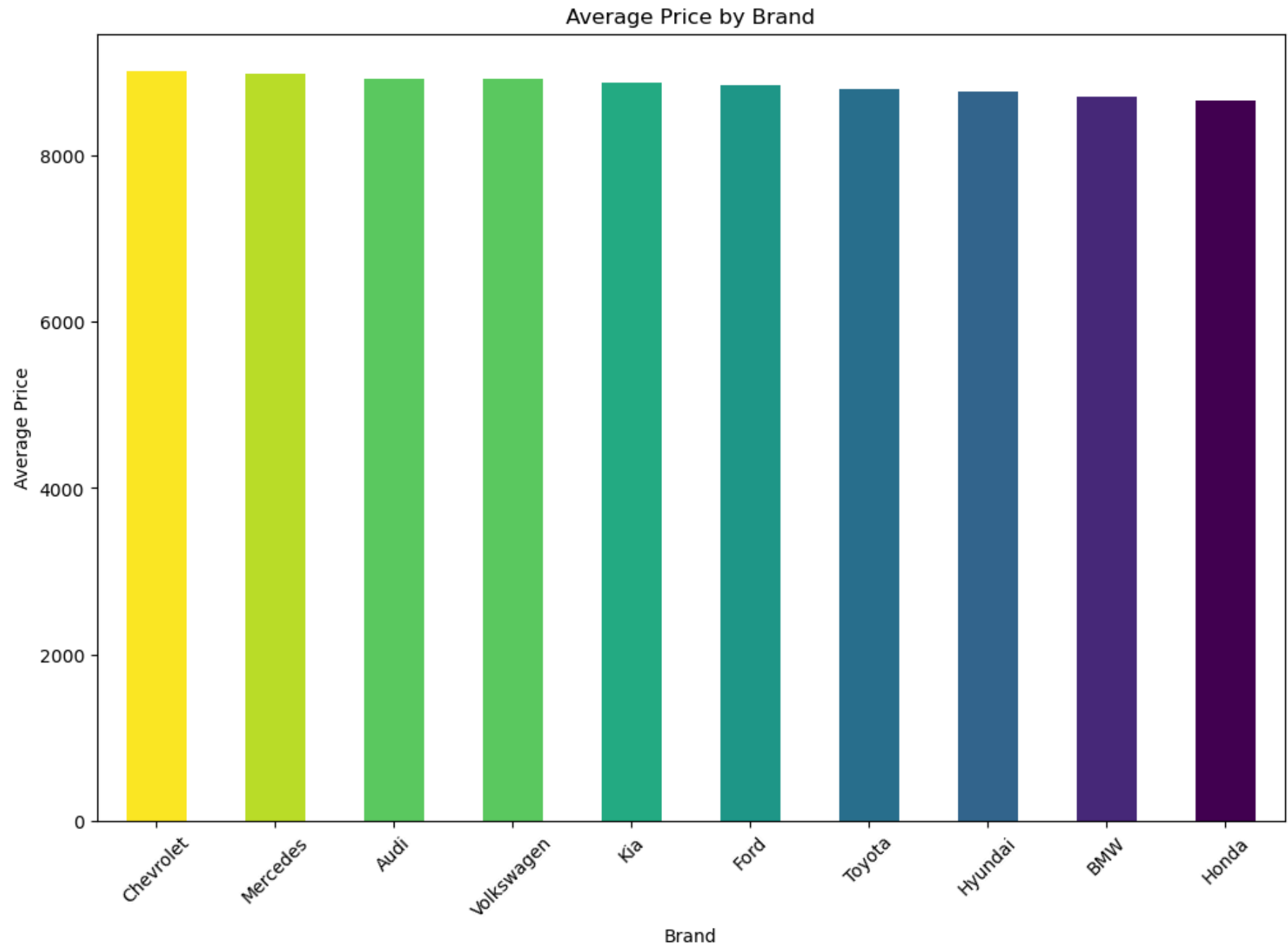
# Step 5: Line Chart (Price Trend by Year)
avg_price_by_year = car_data.groupby('Year')['Price'].mean()
plt.figure(figsize=(10, 6))
plt.plot(avg_price_by_year.index, avg_price_by_year.values, marker='o', color='green')
plt.title('Average Price Trend by Year')
plt.xlabel('Year')
plt.ylabel('Average Price')
plt.grid(True)
plt.show()

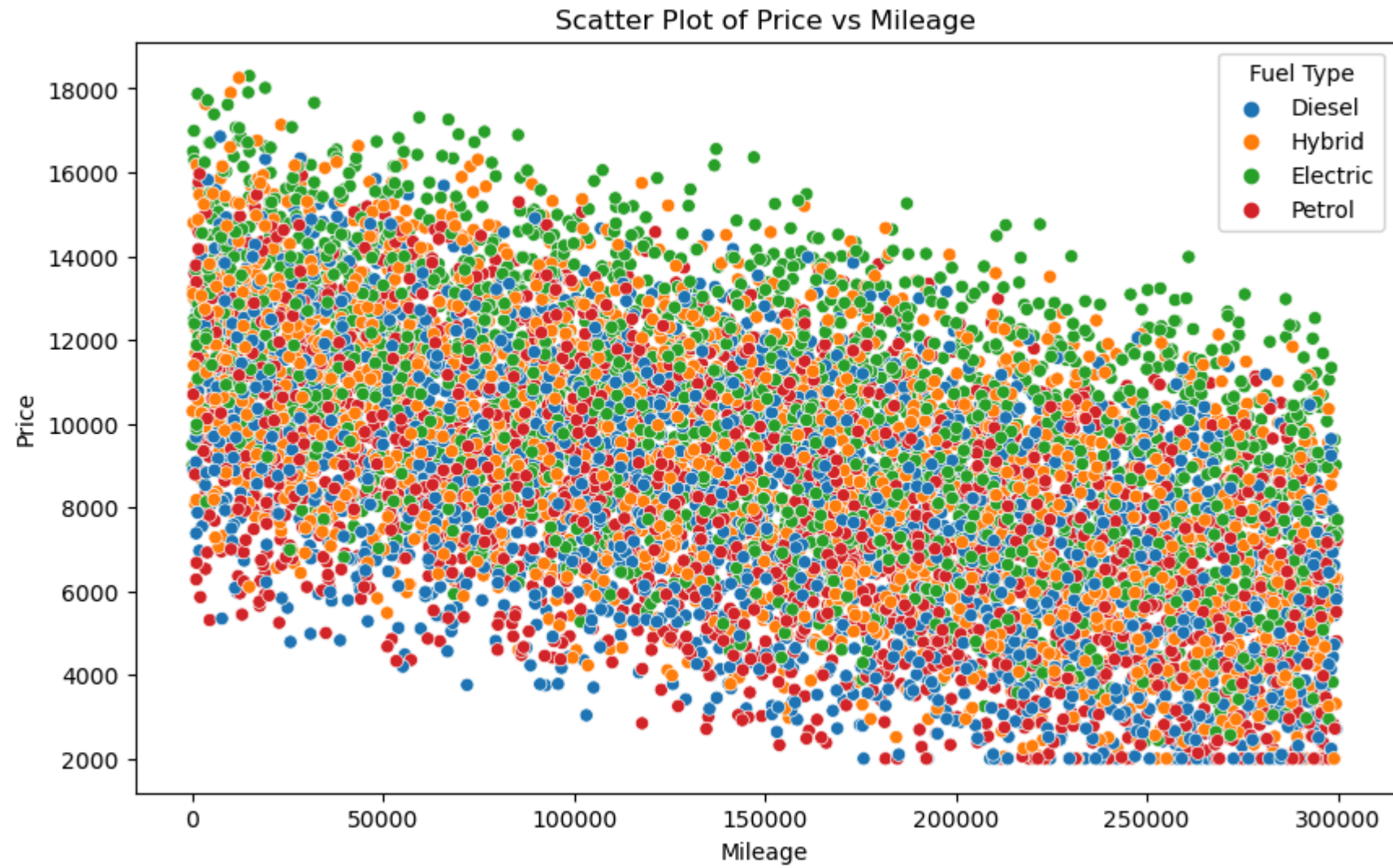
# Step 6: Pie Chart (Distribution of Fuel Types)
fuel_type_counts = car_data['Fuel_Type'].value_counts()
plt.figure(figsize=(8, 8))
plt.pie(fuel_type_counts, labels=fuel_type_counts.index, autopct='%1.1f%%', startangle=140, colors=['gold', 'pink',
plt.title('Distribution of Fuel Types')
plt.show()

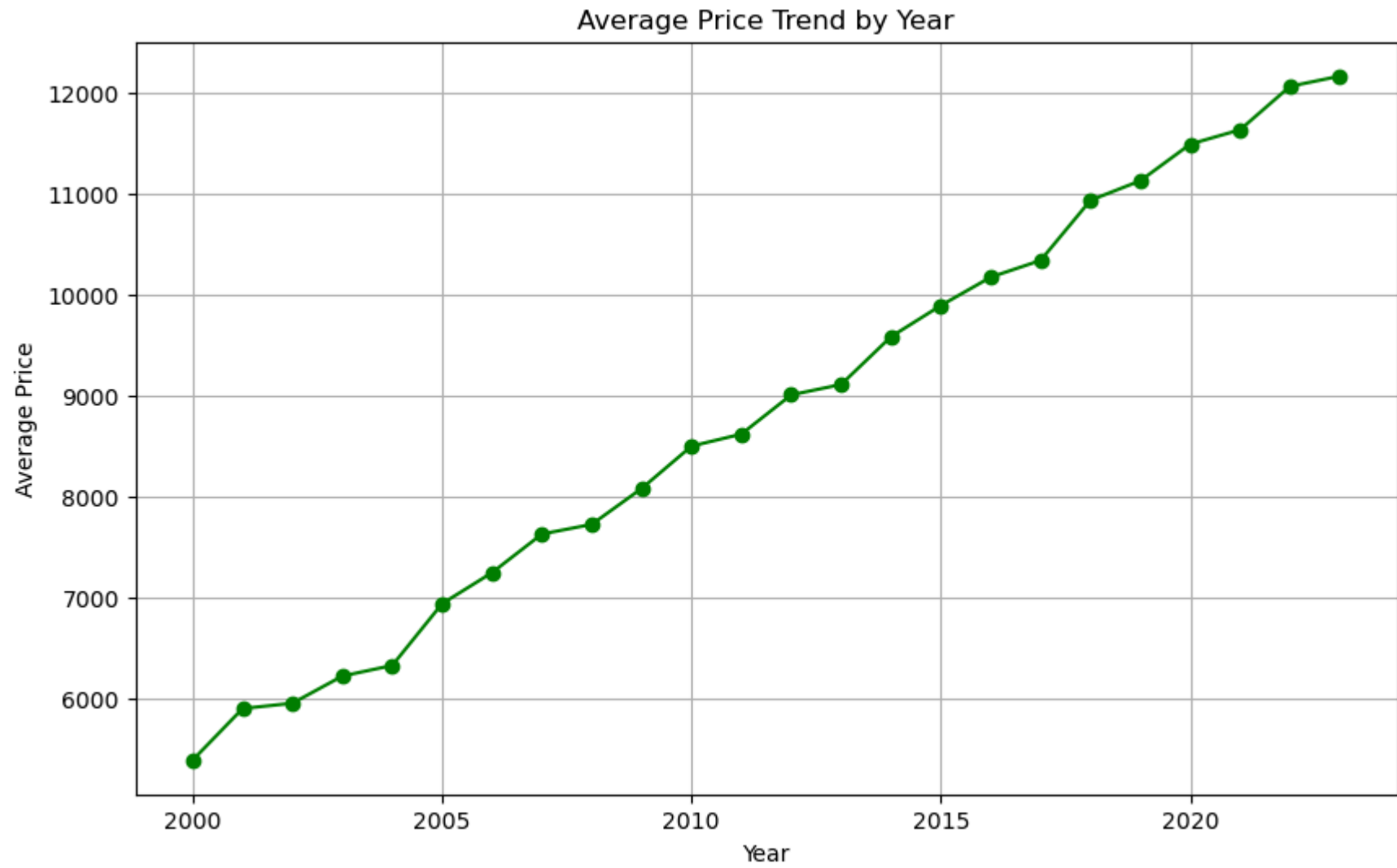
# Step 7: Bar Chart (Number of Cars by Transmission Type)
plt.figure(figsize=(10, 6))
transmission_counts = car_data['Transmission'].value_counts()
transmission_counts.plot(kind='bar', color='lightseagreen')
plt.title('Number of Cars by Transmission Type')
plt.xlabel('Transmission Type')
plt.ylabel('Number of Cars')
plt.xticks(rotation=0)
plt.show()
```

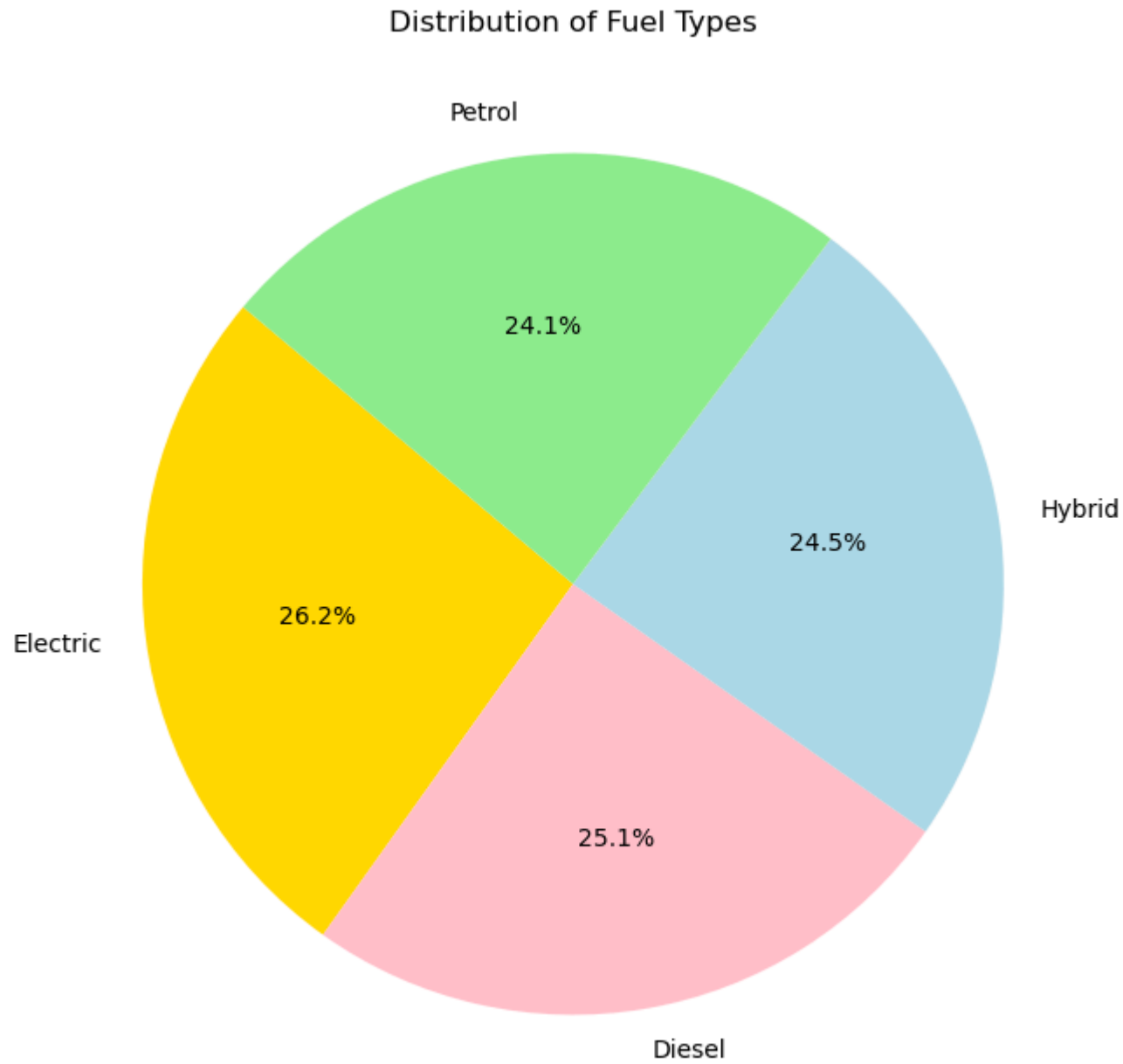


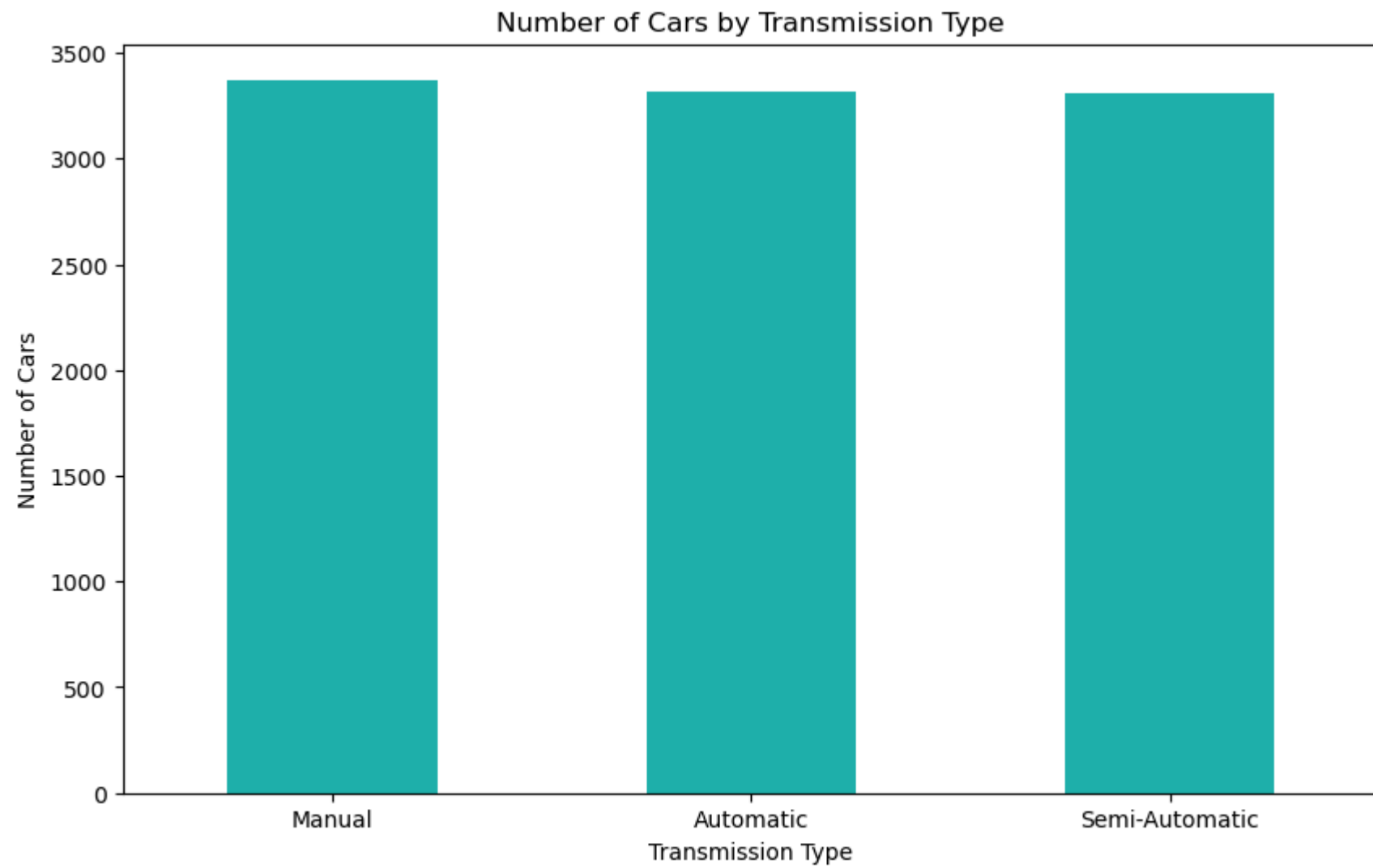
```
/tmp/ipykernel_117/3419390484.py:43: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be removed two minor releases later. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cmap(obj)`` instead.  
cmap = cm.get_cmap('viridis')
```











In []: